

TreeDiff: AST-Guided Code Generation with Diffusion-based LLMs

Yiming Zeng^{1*} · Jinghan Cao^{2*} · Zexin Li³ · Yiming Chen⁴ · Tao Ren⁵ · Zhuochun Li⁵ · Dawei Xiang¹ · Xidong Wu⁵ · Shangqian Gao⁶ · Tingting Yu¹

¹UConn · ²SF State · ³UC Riverside · ⁴NUS · ⁵Pitt · ⁶Florida State · * Equal contribution

First AST-aware masking for DLLMs → **+13.3% on HumanEval+**, narrows gap with autoregressive models

🔗 Motivation

Diffusion-based LLMs (DLLMs) struggle with **syntactic precision** in code generation. Standard random token masking:

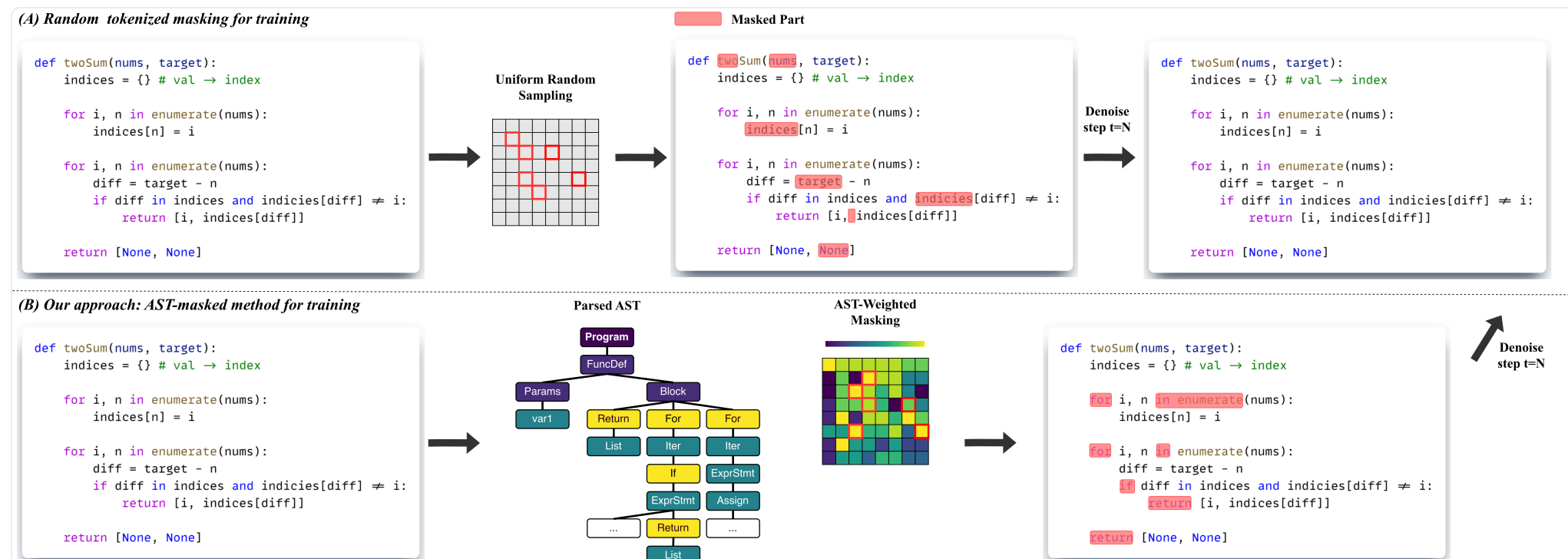
- Ignores syntactic boundaries during denoising
- Fails to capture long-range hierarchical dependencies
- Treats all tokens equally — but they are not

CORE INSIGHT

“For code, noise should not be purely stochastic — structural priors enable DLLMs to learn program-level dependencies.”

🔗 Method: AST-Guided Masking

TreeDiff parses code into an **Abstract Syntax Tree (AST)** and uses it to guide which tokens to mask — targeting structurally critical nodes.



📁 Region-specific Corruption

Prompt Kept intact (semantic conditioning)

Reasoning Random token masking

Code AST-weighted masking

GOAL

Preserve semantic conditioning while injecting structural noise into code regions.

🔗 AST Node Weighting

Category	Examples	Weight	Purpose
Skeletal	Imports, Constants	0.15	Preserve skeleton
Data Flow	Assigns, Calls	0.42	Moderate priority
Conditionals	if, elif, else	0.58	Core logic
Control Flow	for, while, return	0.60	Highest priority

📊 Main Results (Pass@1 %)

13.3%

Relative Gain
HumanEval+ (T=512)

42.1%

HumanEval Pass@1
(vs 39.6% baseline)

Model	T = 256				T = 512			
	HE	HE+	MBPP	MBPP+	HE	HE+	MBPP	MBPP+
Autoregressive Models								
DeepSeek-Coder-33B	50.6	44.5	80.4	70.1	50.6	44.5	80.4	70.1
CodeQwen-7B	51.8	45.7	73.5	60.8	51.8	45.7	73.5	60.8
CodeLlama-13B	42.7	38.4	63.5	52.6	42.7	38.4	63.5	52.6
Diffusion-based LLMs (LLaDA Backbone)								
LLaDA-Instruct	39.6	34.8	51.9	43.1	36.6	31.7	39.4	31.7
+ Random Masking	39.6	35.4	52.9	43.9	38.4	32.3	41.5	32.8
Ours								
+ TreeDiff	42.1	37.2	52.9	44.2	40.2	36.6	42.3	33.3

HE = HumanEval, HE+ = HumanEval+. Autoregressive results from EvalPlus Leaderboard.

⚖️ Ablation: Masking Granularity

HumanEval (HE / HE+)		T=256	T=512
Strategy			
Random Masking		39.6 / 35.4	38.4 / 32.3
AST (Span)		40.9 / 36.6	40.2 / 36.6
AST (Node)		42.1 / 37.2	40.2 / 36.6
MBPP (MBPP / MBPP+)			
Strategy			
Random Masking		52.9 / 43.9	41.5 / 32.8
AST (Span)		51.6 / 42.9	39.7 / 31.5
AST (Node)		52.9 / 44.9	42.3 / 33.3

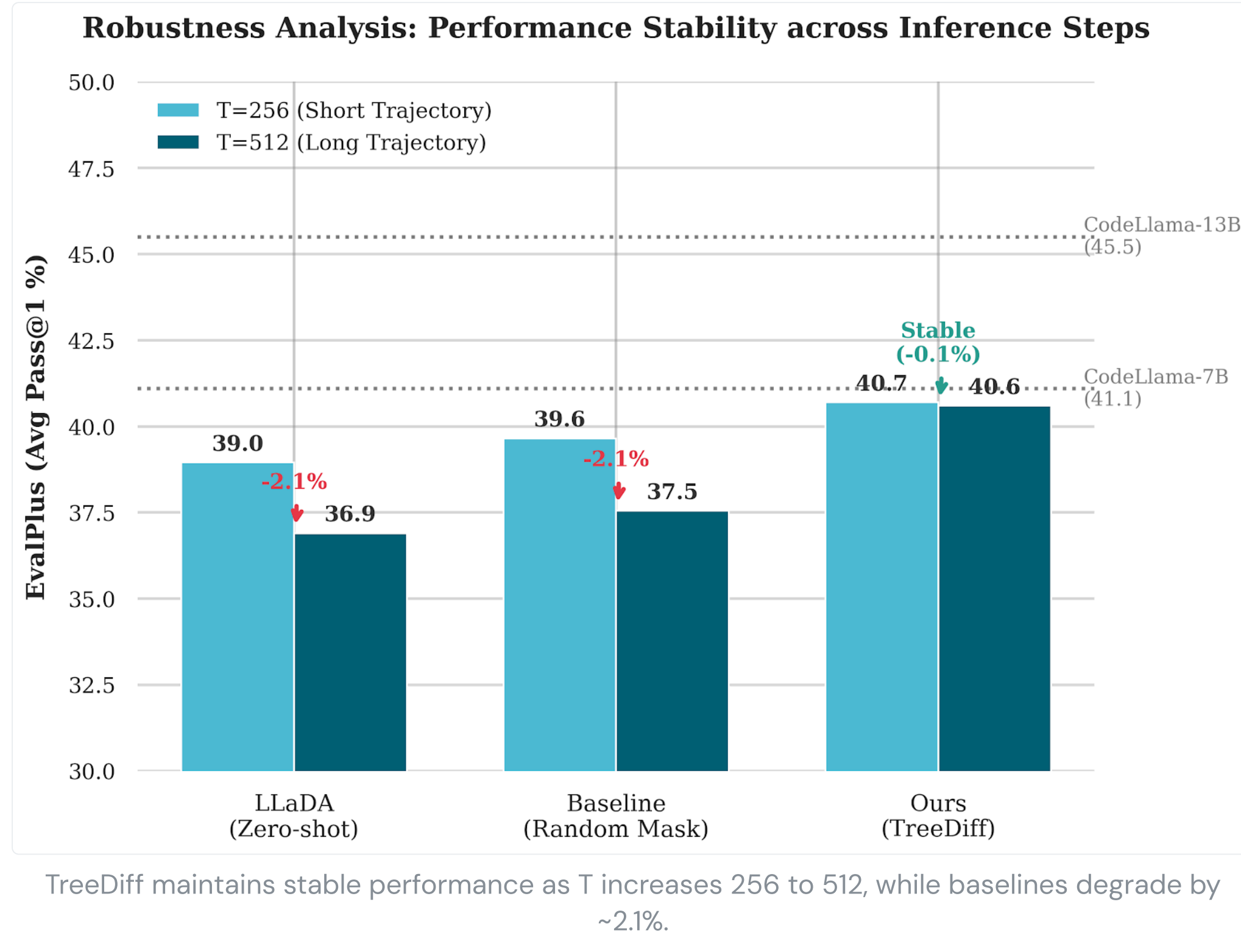
Why Node-level > Span-level?

- **Span masking:** corrupts entire subtrees, removes too much local context
- **Node masking:** targets specific syntactic nodes, preserves surrounding context

🔍 Robustness Analysis

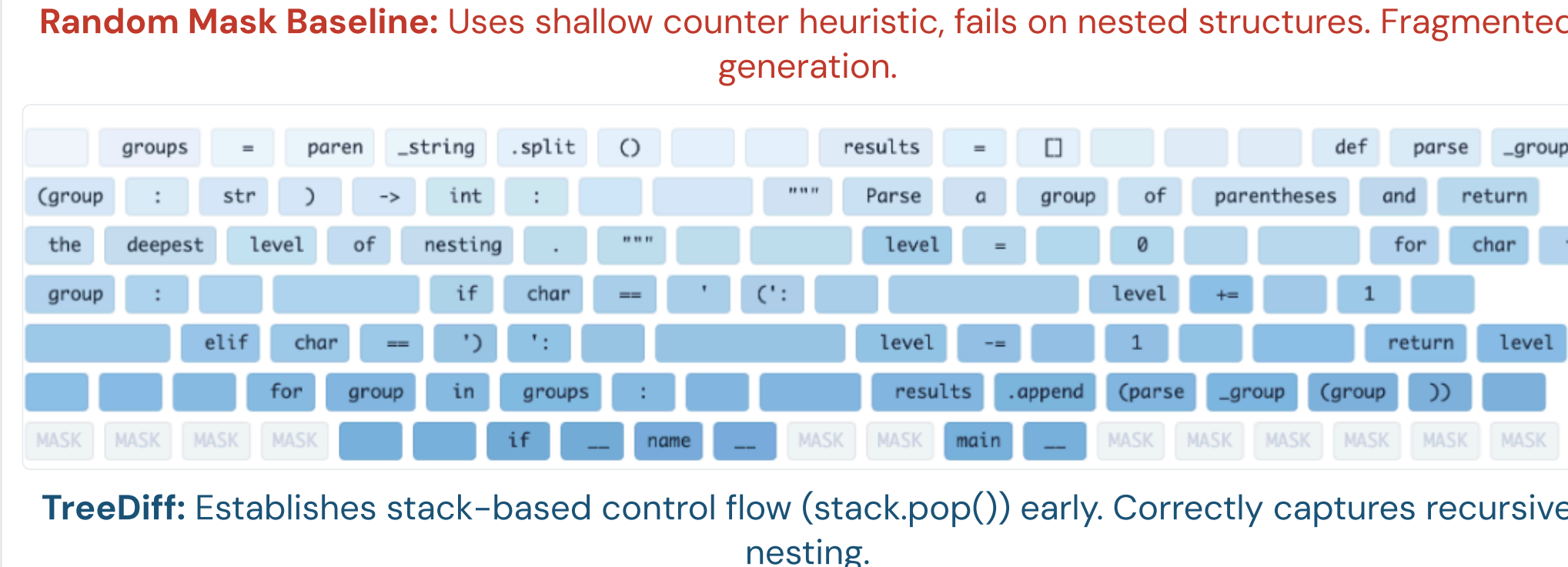
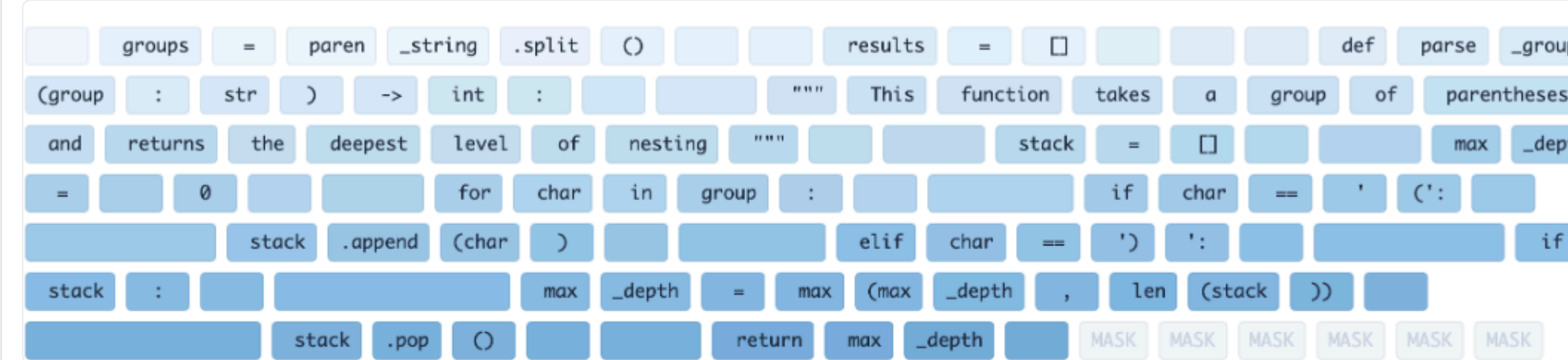
Random masking accumulates syntactic/logical drift at T=512.

TreeDiff remains stable (~0.1%) by anchoring control-flow and skeleton nodes.



🔍 Generation Trajectory Analysis

Task: Max nesting depth of parentheses (HumanEval/6), Step 110/256.



🔍 Generated Code Comparison

HumanEval/6 — Nested parentheses depth:

TreeDiff	Random Masking
for char in group: if char == '(': stack.append(char) max_depth = max(max_depth, len(stack)) elif char == ')': if stack: stack.pop()	count = 0 for char in group: if char == '(': count += 1 # Fails to track # nested peak depth

🔍 Why TreeDiff Works

- Random masking **breaks syntactic anchors** — the model loses structural landmarks
- Span masking **removes too much local context** — entire subtrees vanish
- TreeDiff **masks high-influence AST nodes** while preserving surrounding code structure

🌟 Key Contributions

- **First AST-aware masking** for DLLMs in code generation
- **13.3% relative improvement** on HumanEval+ vs random masking
- Trained on **150K** long code reasoning samples
- Negligible overhead: AST parsing is O(L)

📄 Conclusion

TreeDiff addresses a fundamental limitation: **random masking ignores code structure**. By aligning corruption with AST hierarchy, the model learns programming language compositionality, reconstructing programs that respect grammatical boundaries and capture long-range dependencies.

Impact: Structural priors unlock DLLMs for code — TreeDiff narrows the gap between diffusion and autoregressive paradigms.

Setup: LLaDA-8B backbone · 8x NVIDIA A100 · LoRA (r=64, alpha=128) · 4096 max tokens · LR 5e-5 · Benchmarks: HumanEval+, MBPP+

⚠️ Limitations & Next Steps

- Long iterative denoising can be slow for very long traces (capped at 1024 tokens)
- Current setup uses LoRA due to 8xA100 memory constraints
- Next: faster inference schedules, broader language coverage, stronger backbones

